

The Intermittent Compute Fabric

One Machine Across Many Devices

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-007 v1.0 • series II

October 7, 2026 • cc by 4.0

Abstract — The median technology user now owns five to eight compute devices — phone, laptop, desktop, tablet, watch, console, single-board computer, NAS — containing more aggregate compute, memory, and sensor capacity than most small startups possessed a decade ago. These devices behave as strangers. The only fabric joining them is the cloud round-trip: sync services, push relays, and vendor lock-in, adding latency, cost, and privacy exposure to operations between two machines in the same room. The industry’s answer, continuity features (Apple Handoff/Universal Control/Sidecar, Microsoft Phone Link, Samsung Flow), is closed, per-vendor, and per-feature — a designer’s curated cross-device moment, not a platform. A research line that could have generalized the idea — mobile code offloading (MAUI, CloneCloud, Odyssey, ~2010-2012) — died for reasons that were real then (heterogeneity, security, battery economics, network variability) and have quietly expired (mesh VPNs make peer reachability trivial; mTLS/device identity is solved; CRDT sync engines are mature; NPUs/GPUs are everywhere; local-first culture and the self-hosting renaissance demonstrate demand; and distributed LLM inference is the first workload in a generation that *needs* cross-device placement). We specify the **intermittent compute fabric**: a userspace layer that makes a person’s device set programmable as one logical machine — a global scheduler whose placement objective includes device capabilities, *energy state*, and *predicted connectivity*; a capability-based delegation protocol (a laptop app borrowing the phone’s camera, a phone borrowing the desktop’s GPU); a partition-tolerant task semantics with explicit intermittent execution; and an ambient CRDT state layer replacing cloud sync for fabric-resident data. We define the failure model (device sleep as process migration; partition as scheduling input; revocation as preemption), state the formal properties (task completion under partition, bounded-energy placement, capability safety), design the architecture and task-spec DSL, present an evaluation methodology (2–3 device prototype, partition traces, latency-vs-cloud and energy-per-job measurements), and confront the strongest objections: why offloading died, why vendors will never ship it open, why apps won’t target a mesh, and who maintains it. The strategic claim: the fabric is the missing *host* for the local-AI era — private, distributed inference over devices people already own — and the open window is 2–3 years before three closed implementations harden into permanence.

p-rick working paper P-007 · series II (systems) · draft 1.0

The median technology user now owns five to eight compute devices — phone, laptop, desktop, tablet, watch, console, single-board computer, NAS — containing more aggregate compute, memory, and sensor capacity than most small startups possessed a decade ago. These devices behave as strangers. The only fabric joining them is the cloud round-trip: sync services, push relays, and vendor lock-in, adding latency, cost, and privacy exposure to operations between two machines in the same room. The industry’s answer, continuity features (Apple Handoff/Universal Control/Sidecar, Microsoft Phone Link, Samsung Flow), is closed, per-vendor, and per-feature — a designer’s curated cross-device moment, not a platform. A research line that could have generalized the idea — mobile code offloading (MAUI, CloneCloud, Odyssey, ~2010-2012) — died for reasons that were real then (heterogeneity, security, battery economics, network variability) and have quietly expired (mesh VPNs make peer reachability trivial; mTLS/device identity is solved; CRDT sync engines are mature; NPUs/GPUs are everywhere; local-first culture and the self-hosting renaissance demonstrate demand; and distributed LLM inference is the first workload in a genera-

tion that *needs* cross-device placement). We specify the **intermittent compute fabric**: a userspace layer that makes a person’s device set programmable as one logical machine — a global scheduler whose placement objective includes device capabilities, *energy state*, and *predicted connectivity*; a capability-based delegation protocol (a laptop app borrowing the phone’s camera, a phone borrowing the desktop’s GPU); a partition-tolerant task semantics with explicit intermittent execution; and an ambient CRDT state layer replacing cloud sync for fabric-resident data. We define the failure model (device sleep as process migration; partition as scheduling input; revocation as preemption), state the formal properties (task completion under partition, bounded-energy placement, capability safety), design the architecture and task-spec DSL, present an evaluation methodology (2–3 device prototype, partition traces, latency-vs-cloud and energy-per-job measurements), and confront the strongest objections: why offloading died, why vendors will never ship it open, why apps won’t target a mesh, and who maintains it. The strategic claim: the fabric is the missing *host* for the local-AI era — private, distributed inference over devices people already own — and the open window is 2–3 years before three closed implementations harden into permanence.

1 Introduction

Consider a completely ordinary evening: a person with a phone, a laptop, and a desktop (with a GPU) wants to run a 7B-parameter language model privately. The phone cannot fit it; the laptop runs it slowly; the desktop runs it well but is across the room. Industry’s answer is buy a cloud subscription, or carry the workload by hand (copy the file, run it there, copy it back) — the ritual everyone performs daily across documents, media, builds, and model weights. The devices are islands connected through a third party that bills by the month and reads by the byte. The absurdity is structural, not incidental: nothing in the consumer stack lets one machine *ask another machine to do work*.

The industry solved the trivially visible slices. Apple’s continuity stack (Handoff, 2014; AirDrop; Sidecar, 2019; Universal Control, 2022) is polished — and hermetic: Apple-to-Apple, curated feature-by-feature, closed to third-party developers as a platform. Microsoft Phone Link and Samsung Flow mirror the pattern on the other duopoly. KDE Connect and LocalSend prove community appetite for open cross-device behavior (file transfer, input sharing) without a compute layer. None of these — vendor or community — offers the general capability: *a programming model in which the device set is one schedulable machine*.

There is a name for that idea, and it is old. Distributed operating systems research (Sprite, LOCUS, Amoeba, MOSIX single-system-image work, Plan 9’s resource-centric design) built exactly this for the machines of its era — pools and clusters. When computing went personal and mobile, the research line forked into mobile *code offloading* (Odyssey’s application-aware adaptation; MAUI’s energy-optimal method migration; CloneCloud’s VM-based partitioning), which flourished 2008–2013 and then died. The field’s honest post-mortems list the killers: device heterogeneity, security of remote execution, battery economics, and the killer of all of them — the arrival of cloud APIs that made offloading someone else’s problem. Meanwhile the actual substrate of personal computing kept compounding: every device became multi-core with a heterogeneous accelerator, always-connected over encrypted mesh-capable networks, owned by one identity, and — the recent turn — running local AI workloads that are large, divisible, and placement-sensitive.

This paper specifies the system that the substrate now permits: the intermittent compute fabric. §3 explains why the offloading literature died and why its death conditions expired; §4 surveys the modern landscape’s gaps; §5–6 specify the fabric — scheduler, capability protocol, task semantics, state layer; §7 designs the evaluation; §8 confronts the objections; §9 argues the AI-era window.

2 The problem: five islands owned by one person, mediated by a billing meter

Enumerate the ordinary cross-device operations people perform daily, and their costs:

- **Carry** (move a file or state by hand: USB, chat-to-self, email-to-self, sync folder): seconds to minutes, error-prone, version-ambiguous.
- **Cloud round-trip** (everything else): adds WAN latency, subscription cost, privacy exposure, and offline failure to operations between devices in the same room.
- **Vendor moment** (AirDrop, Handoff, Phone Link): works — exactly and only — for the curated features, vendors, and directions the vendor blessed.

What does not exist is the *primitive*: `run(task, constraints) -> somewhere in my device set`, subject to my placement policy. Its absence shows up as:

Capability stranding. The desktop GPU idles while the phone struggles; the phone’s sensors idle while the laptop’s video call wants them; the NAS stores terabytes while the cloud bills for gigabytes. The capability graph is rich; the reachability graph is feudal.

Energy myopia. A compute job runs on the phone in the pocket at 15W from battery when the plugged-in laptop at 65W wall-power sits idle in the same bag. Offloading research showed order-of-magnitude energy savings from placement; consumer platforms expose none of it.

Offline fragility. Trains, planes, dead zones, roaming: the cloud-mediated fabric fails exactly when device-local resources are abundant. Partition tolerance is a *cloud* property in today’s stack; it should be a *local* property.

Privacy deadweight. Data that never needs to leave the house takes the cloud round-trip anyway, because the local path is unprogrammable. The local-first movement (Ink & Switch, CRDT sync engines) has rebuilt *state* local-first; *computation* remains cloud-feudal.

The aggregate is a user who owns a distributed computer and uses it as five terminals on a metered bus.

3 Why offloading died, and why the death expired

The mobile offloading literature (surveyed in Kumar & Lu’s energy analysis, and the Mach/Becvar line) was technically sound: MAUI (MobiCom 2010) minimized energy via method-level offloading over WiFi; CloneCloud (EuroSys 2011) partitioned computation via VM migration; Odyssey (SOSP 1997 / MobiCom 2002) introduced application-aware adaptation between local and remote execution; Satyanarayanan’s cloudlet line pushed the execution substrate toward the edge. The failure conditions, as the field’s own retrospectives tell it:

1. **Reachability was unsolved.** NATs, dynamic IPs, and absent device identity made “call my other machine” a network research problem per session. *Expired:* WireGuard-class tunnels (NDSS 2017), Tailscale-style mesh coordination (2019–), and mDNS make any device set addressable by stable identity over an encrypted mesh with zero configuration. This is the single largest substrate change: the fabric’s network layer is now a weekend, not a thesis.
2. **Trust and code mobility were open wounds.** Executing foreign code on your laptop was an attack surface; carrying credentials between devices was unsolved. *Expired:* per-device identities with mutual TLS, signed task bundles, sandboxed runtimes (WASM, containers) — remote execution is now routine engineering (it is, literally, what CI clouds sell).
3. **Heterogeneity made binaries fragile.** *Expired by neutralization:* WASM and container images give portable execution units; and the fabric can place by capability-match rather than by architecture-match (the GPU job goes where a GPU is; it does not need to run *everywhere*).

4. **Battery economics favored the cloud.** Offloading saved battery only over WiFi; cellular made it a wash; and the cloud made someone else pay the joules. *Expired:* the AI workload inversion — the user *owns* the FLOPs locally (the desktop GPU was bought for this), and the question is placement among owned resources, not rent-versus-battery. The fabric is not renting compute; it is *scheduling owned compute*.
5. **No workload demanded it.** Document sync — the dominant cross-device need — never needed remote execution, only state replication. *Newly false:* local LLM inference is divisible (layer/pipeline partitioning — the Petals line demonstrated collaborative serving over consumer GPUs; the EdgeShard line formalized edge partitioning), placement-sensitive (NPU here, memory there), and privacy-shaped (do not send my context to a cloud). For the first time since 2012, there is a workload that *wants* the fabric.

The honest summary: offloading died because the substrate and the workload were wrong, and both have now inverted. What has not expired is the graveyard’s effect: the field stopped looking, and the vendor stack filled the space with closed moments.

4 The modern landscape: every layer exists, no integration

- **Network:** Tailscale/WireGuard, Nebula, ZeroTier — personal meshes at production maturity; head-scale for self-hosting.
- **Identity:** per-device keys, mTLS, capability-token patterns; the missing piece is *user-scoped* federation (one identity binding *this person’s* devices) — an engineering task, not research.
- **State:** CRDT engines (Automerge, Yjs) and sync systems (Syncthing’s partition-tolerant P2P file sync — a fabric-native primitive hiding in a file tool).
- **Execution:** WASM runtimes, OCI containers, SSH, remote APIs; Planet-scale precedent for scheduling across owned clusters (k3s on homelab Raspberry Pis is a mass hobby).
- **Workload:** local-first AI (llama.cpp-class runtimes, NPU runtimes on every phone), Petals-style distributed serving, split-computing surveys.
- **UX precedents:** Universal Control (one input across devices), KDE Connect, AirDrop; the interaction grammar exists feature-by-feature.
- **Community demand:** *r/selfhosted*, Home Assistant’s growth (a *de facto* fabric for sensors, running on owned hardware, with hundreds of integrations), the homelab renaissance (CasaOS/Umbrel-class platforms packaging server-ness for consumers).

What does not exist anywhere: the *scheduler + capability protocol + task semantics* that turn these layers into a programmable machine. Home Assistant automates the smart home; the mesh VPN moves packets; the CRDT syncs documents; nothing lets a program say “run this where the FLOPs are, borrow the camera where the camera is, keep state where I am, and survive the phone sleeping.”

5 The fabric: model and semantics

5.1 The fabric abstraction

A **fabric** $F = (D, E, S)$: a finite set of devices D , a reachability/energy/capability envelope E , and a state layer S . To the programmer, F is a single machine with heterogeneous cores (the devices), a capability registry (what each device exposes — GPU/NPU classes, memory tiers, sensors, actuators, storages), and an unusual failure model: *cores sleep on their own schedule, join and leave unpredictably, and have energy budgets of their own*.

5.2 Tasks and the task-spec DSL

A **task** is a signed, content-addressed bundle: code (WASM/container/script), a resource contract (in the P-004 sense: CPU/GPU/memory floors, energy preference, deadlines), input references (into S or the ledger of P-005 — the papers compose), and output semantics. Tasks are declared with three placement dimensions: **affinity** (must/should/prefer: data locality, capability match), **mobility** (static / migratable / resumable), and **criticality** (interactive latency class vs. background throughput). The DSL is deliberately boring YAML:

```
task: transcribe-meeting
code: wasm://sha256:... (whisper-wasm)
inputs: [state://recordings/2026-10-07]
resource: cpu: 2, memory: 2Gi, accelerator: npu|gpu, energy: wall-preferred
placement: affinity: data-local, mobility: resumable, criticality: background
deadline: tonight
```

5.3 The failure model — the fabric’s actual contribution

Every distributed system is defined by its failure model. The fabric’s is unusual and must be first-class:

- **Device sleep = process migration.** A phone locking is not an error; it is a *scheduled* unavailability. The scheduler’s connectivity predictor (below) treats sleep as an input; resumable tasks checkpoint and migrate. Formally: preemption with guaranteed resumption on wake or relocation — semantics borrowed from process migration literature, hardened by CRDT checkpoints.
- **Partition = scheduling constraint.** The fabric assumes partitions are the common case (devices leave WiFi range constantly). Placement under predicted connectivity: a task with a 2-hour deadline whose inputs sit on the desktop *may* be placed there only if the predicted reachability window covers the task duration; otherwise it is *degraded* to a smaller-device variant (P-004 contract) or deferred. The formal object: a Markov-style reachability model per device-pair, learned from observed join/leave patterns — the same prediction machinery attention scheduling uses for humans (P-006), applied to devices.
- **Revocation = preemption.** Capabilities are tokens with expiry and revocation; a device leaving the fabric (its owner turned it off, revoked trust) preempts its tasks — with the same resumption guarantees.

Three properties define correctness: (i) **completion under partition:** a task admitted with deadline d completes by d if at least one qualifying device remains reachable to its input state — completion is *partition-tolerant by construction* (CRDT state replicates with the task’s durability class); (ii) **bounded-energy placement:** admitted placements are within factor $(1+\epsilon)$ of the optimal energy/capability assignment under the predicted model — placement is an online optimization with competitive-ratio discipline, not a heuristic; (iii) **capability safety:** code executing on a device holds exactly the capabilities its task-spec named, granted by tokens, revocable, auditable (a device’s ledger records what ran and what it touched — P-005’s ledger, federated).

5.4 The scheduler

A per-device daemon, leaderless by default (each device schedules its *own* submissions against the shared envelope; a quorum-elected coordinator for global jobs). The placement solve is constrained optimization: minimize a weighted objective (energy, latency, privacy exposure, wear) subject to resource contracts, capability match, connectivity prediction, and user policy (the policy DSL: “nothing of mine runs on the work laptop; GPU jobs only on wall power; my data never leaves my devices”). The objective weights are the user’s — the fabric is explicitly *policy-first*, because the fabric’s owners are not operators of a datacenter; they are people with preferences. Inputs: device capability registry (static), energy state (battery %, charging, thermals — P-004’s pressure signals, again shared), connectivity prediction (learned), load telemetry. Output: placement + migration schedule + checkpoint plan.

5.5 The state layer

S is CRDT-based ambient state with per-key durability classes: ephemeral (queue-like), session (device-following — state migrates with the user’s active device), durable (replicated to $\geq k$ devices of the owner’s choosing; optional cloud as just another device with bad privacy metadata). This is the local-first thesis extended from documents to *all fabric state*, and it is what makes tasks location-free: inputs are addressed by name, not by device; the state layer ensures the data is where the task lands (or the task goes to the data — affinity decides, energy prices the choice).

5.6 Capability delegation

The fabric’s most user-visible magic: device A executing code that borrows device B’s capability (the laptop’s video call borrowing the phone’s camera and microphone — Universal Control’s feel, as an API). Delegation is a scoped capability token: {capability, grantor-device, grantee-task, expiry, data-policy}; revocation is instant; audit lives in both devices’ ledgers. The protocol handles the awkward middle: who composites the video, where the model runs, how the streams encrypt (SRTP-style end-to-end, keys never leaving the pair). This is the piece with no open precedent anywhere — the vendor continuity features are its closed ancestors, and opening it is the fabric’s platform bet.

6 Architecture

Userspace everywhere (no OS vendor dependency — the strategic requirement, §8): per-device fabric daemon (Go/Rust, single static binary); device identity = keypair, user federation = signed device-set membership (the “one person” binding, verified out-of-band once); mesh transport = existing WireGuard-class tunnels, discovered via Tailscale/self-hosted coordination or pure LAN mDNS; execution = WASM runtimes + OCI + ssh-compatible executors with a capability gate; state = embedded CRDT engine (Automerge-class) with storage on local disks; scheduler = the optimization above; policy = the user-facing DSL and console; ledgers = per-device, P-005-compatible. The fabric composes with the other series-II papers deliberately: resource contracts from P-004 govern task behavior under device pressure; P-005 ledgers audit what ran where on what data; P-006’s attention broker is the *only* sanctioned channel for fabric-initiated human interruption (agents and schedulers submit attention requests, they never buzz the human raw). Three papers, one design language: contracts, ledgers, brokers.

7 Evaluation design

Prototype on three real devices (Android phone, x86 laptop, GPU desktop) and a synthetic 8-device fleet in simulation. Measurements:

- **Completion under partition:** replayed 72-hour reachability traces (real join/leave logs from three devices), injected partitions; completion rate and deadline violations by durability class; the baseline is “cloud path + offline = fail.”
- **Latency and energy per job:** representative tasks (document processing, local LLM inference — layer-partitioned across phone NPU + desktop GPU, media transcode) vs. cloud round-trip (latency, joules via device power meters, cost in currency).
- **Capability delegation:** task-level benchmarks of the camera/GPU borrow protocol (setup latency, throughput, revocation propagation).
- **Overhead:** daemon CPU/memory footprint on the *phone* (the binding constraint), mesh chatter under 8 devices.

- **Placement quality:** regret vs. offline-optimal placement on synthetic workloads with known optima (competitive-ratio validation of §5.4).

8 Objections, confronted

“This died before; it will die again.” §3 is the answer: enumerate the death conditions, verify each expired, and note the one that did *not* — no workload demanded it. The workload now exists (§9), and this time the joules are already bought.

“Vendors will never ship it open — and closed continuity wins by default.” True that vendors will not ship it; false that it needs them. The fabric is userspace over substrate every OS already exposes (VPN, WASM, sensors, ledgers). The strategic claim is Linux-desktop-then-enthusiast-then-everywhere — the Home Assistant trajectory (a userspace fabric for sensors that reached millions without any vendor’s blessing) is the existence proof that consumer device fabrics can grow from below. The concession: the phone duopoly will fight background execution and sensor access on their terms; the fabric’s answer is partial presence (the phone joins when foregrounded/charging — which the connectivity model treats as normal, not broken).

“Why would apps target a mesh no one has?” They will not, at first — and the fabric does not need them to. Its first consumers are the *user’s own* automations: scripts, agents, sync jobs, media pipelines (the same tasks people currently hand-carry). The app story arrives through delegation (§5.6): an app on any one device gains the whole fabric’s capabilities *without targeting the fabric* — the laptop app “just” has a camera and a GPU, because the fabric mounts them. That is the same trick Universal Control plays on input; generalized, it is the platform.

“Security: remote code across personal devices is an attack surface.” Capability tokens bound to signed task bundles, sandboxed executors, per-device ledgers, revocation — the machinery that makes cloud CI safe to run on other people’s machines, pointed at your own. The honest concession: *any* multi-device system increases attack surface; the fabric’s design goal is that it is strictly safer than the status quo it replaces (files carried by chat apps and emailed to self, with no audit trail at all).

“Who maintains it?” The graveyard’s best objection. Answer: the maintenance burden must be *shaped* small — a static daemon, boring protocols, and maximal reuse of boring infrastructure (Wire-Guard, WASM, CRDTs) — and the sustainability model is the Home Assistant one: a core maintained by a foundation-class entity, integrations by communities. If the design’s honest maintenance estimate exceeds that shape, it should not be built. That discipline (falsifiable complexity budget) is stated as a design constraint, not a hope.

9 The window: local AI and the two-to-three-year gap

Distributed inference over owned devices is the fabric’s killer workload and its deadline. The Petals line proved collaborative LLM serving works over the open internet with consumer GPUs; the edge-partitioning literature (EdgeShard and successors) formalized layer-wise placement on constrained devices; every phone vendor ships NPUs; every laptop ships NPUs; and the privacy argument for *not* sending context to clouds has never been stronger, or more widely understood. Three closed implementations are visibly converging on this space from above (vendor continuity + vendor AI stacks): when they land, the personal-device mesh as a *programming model* will be permanently annexed into walled gardens — the same annexation that happened to synchronization (iCloud/Dropbox replaced the open local-sync world) and is one feature-release away from happening to device-adjacent compute. The open fabric’s chance is to exist, with real scheduling semantics and an open protocol, before the annexation — so that “your devices, one machine” is a spec anyone can implement, not a marketing phrase one vendor owns.

10 Conclusion

The personal device set is the last great un-programmed computer: more compute than a startup, more sensors than a lab, one owner, one identity, and an architecture of five islands on a metered bus. The research line that dreamed of programming it died honorably in 2012 for reasons the substrate has since erased: reachability, identity, portability, and — now — workload. What remains is to specify the machine that the pieces compose into: a scheduler that treats sleep and partition as inputs rather than errors, a capability protocol that turns vendor moments into open APIs, a state layer that makes location an implementation detail, and a policy surface that keeps the owner — not the platform — as the placement authority. The fabric is buildable in userspace today, by the community that already builds Home Assistants and mesh VPNs and homelabs, for the workload that is already arriving. The islands are real, the sea between them is drained, and the bridges are unbuilt. This paper is the engineering survey for the first one.

Disclosure

p-rick is an independent research program; no vendor has commissioned or reviewed this work; the author has no financial interest in cited systems. The fabric composes with P-004 (resource contracts) and P-005 (provenance ledgers); P-006 (attention scheduling) defines the fabric’s human-interruption discipline. The cross-references are design-level, not commercial.

References

1. Cuervo, E., et al. (2010). *MAUI: making smartphones last longer with code offload*. ACM MobiCom.
2. Chun, B.-G., et al. (2011). *CloneCloud: elastic execution between mobile device and cloud*. ACM EuroSys.
3. Flinn, J., & Satyanarayanan, M. (2002). *Energy-aware adaptation for mobile applications*. ACM MobiCom (the Odyssey application-aware adaptation line; Noble & Satyanarayanan, SOSP 1997, *Odyssey*, the founding design).
4. Satyanarayanan, M., et al. (2009). *The case for VM-based cloudlets in mobile computing*. IEEE Pervasive Computing.
5. Kumar, K., & Lu, Y.-H. (2010). *Cloud computing for mobile users: can offloading computation save energy?* IEEE Computer. / Mach, P., & Becvar, Z. (2015). *Mobile edge computing: a survey on architecture and computation offloading*. ACM Computing Surveys.
6. Ousterhout, J., et al. (1988). *The Sprite network operating system*. IEEE Computer / Barak, A., & La’adan, O. (1998). *The MOSIX single-system image*. IEEE Concurrency / Tanenbaum, A., et al. (1990). *Amoeba line* / Popek, G., et al. (1981). *LOCUS line*.
7. Pike, R., Presotto, B., Thompson, K., et al. (1990–1991). *Plan 9 from Bell Systems* (the resource-as-file design); 9P filesystem protocol (Linux v9fs heritage).
8. Donenfeld, J. A. (2017). *WireGuard: next generation kernel network tunnel*. NDSS. / Denton, A. (2019–). *Tailscale engineering blog* (mesh coordination over WireGuard). / Slack (2019). *Nebula*.
9. Kleppmann, M., Wiggins, A., et al. (2019). *Local-first software: you can own your data, in spite of the cloud*. Ink & Switch TR. / Shapiro, M., et al. (2011). *Conflict-free replicated data types*. SSS. / Kleppmann (2022–). *Automerger* / Wehmeyer, K. (2021–). *Yjs*.
10. Syncthing project (2013–). *Syncthing — partition-tolerant peer-to-peer file synchronization* (the fabric-native primitive hiding in a file tool).
11. Borzov, V., et al. (2023). *Petals: collaborative inference and fine-tuning of large models*. NeurIPS (distributed serving over consumer GPUs).

12. EdgeShard line (2024—). *EdgeShard: LLM inference over edge devices via layer-wise partitioning* and successors (split-computing surveys: Matsubara et al., 2022).
13. Apple documentation (2014–2022): Handoff/Continuity, Sidecar, Universal Control. / Microsoft *Phone Link* / Samsung *Flow* documentation and reviews (2022–2025).
14. KDE Connect project; LocalSend project (open cross-device precedents, no compute layer).
15. Home Assistant project (2013–) and smart-home self-hosting adoption line (the userspace-fabric existence proof).
16. CasaOS / Umbrel / Start9 (2022–2025): the consumer self-hosting renaissance.
17. Rekimoto, J. (1997). *Pick-and-drop: a direct manipulation technique for multiple computer systems*. ACM UIST (the founding cross-device interaction citation).
18. k3s / homelab kubernetes-on-Raspberry-Pi practice (2020–2025) — community scheduling over owned clusters.
19. Zhu, Y., et al. capability-based delegation literature: Dennis & Van Horn (1966), *Programming semantics for multiprogrammed computations* (the capability model’s origin) — the fabric’s security ancestry.
20. Mark, Weiser (1991) — *The computer for the 21st century* (ubiquitous computing vision; the fabric is an answer to it, 35 years late).